

■ FSDDL Oral Exam Notes

Full Stack Web Development Lab · All 10 Experiments · TEI Ichalkaranji

EXPT 01

TypeScript — Basic Programming Concepts

AIM

Demonstrate basic building blocks of TypeScript: Types, Functions, Interfaces, Classes, Inheritance.

THEORY

TypeScript is a superset of JavaScript. Every valid JS is valid TS. It adds static typing — type errors are caught at compile time, not runtime. It transpiles to plain JavaScript using the tsc compiler.

KEY CONCEPTS

- **Types:** boolean, number, string. Explicit: `let name: string = "A"`. Implicit: TS guesses from value.
- **any:** Disables type checking. Avoid it — defeats the purpose of TS.
- **Function:** Parameters and return type are typed. Return `void` means no return value.
- **Interface:** Defines a contract/shape. Only used for type-checking — NOT compiled to JS. Known as 'duck typing'.
- **Class:** Blueprint for objects. Supports constructors, access modifiers (public/private), methods.
- **Interface Inheritance:** extends keyword. Can extend multiple interfaces.

CODE REFERENCE

```
npm install -g typescript    # install TS globally
tsc test.ts                 # compile TS → JS
node test                   # run compiled JS
```

KEY TERMS

[Superset of JS] [Static Typing] [Transpilation] [Interface] [Duck Typing] [tsc compiler]

LIKELY ORAL QUESTIONS

Q: What is the difference between TypeScript and JavaScript?

TS is a superset of JS with static typing. JS is loosely typed — type errors appear at runtime. TS catches them at compile time.

Q: What is an interface in TypeScript?

An interface defines a structure/contract a class or object must follow. It is only used for type-checking and is NOT compiled into JavaScript.

Q: What is duck typing?

If an object has all the properties an interface expects, TS treats it as that type — regardless of what it was explicitly declared as.

Q: What does 'any' type do?

It disables type checking for that variable, allowing any type to be assigned. It's like writing plain JavaScript.

AIM

Set up an Angular project and understand its basic building blocks and folder structure.

THEORY

Angular is an open-source frontend framework by Google. It builds Single Page Applications (SPAs) and Progressive Web Apps (PWAs) using a component-based architecture.

KEY CONCEPTS

- **Component-Based:** UI broken into reusable self-contained components.
- **Two-Way Data Binding:** Model ↔ View sync automatically.
- **Dependency Injection:** Services injected automatically, loosely coupled.
- **Directives:** Extend HTML behaviour (ngFor, ngIf, ngModel).
- **Routing:** Built-in router for navigation between views/URLs.
- **HTTP Client:** Built-in module for REST API calls.
- **Key files:** app.component.ts (logic), app.component.html (template), app.config.ts (providers).

CODE REFERENCE

```
npm install -g @angular/cli    # install Angular CLI
ng new my-angular-app          # create new project
cd my-angular-app
ng serve                        # runs at http://localhost:4200
```

KEY TERMS

[SPA] [Component] [Angular CLI] [ng serve] [Google]

LIKELY ORAL QUESTIONS

Q: What is Angular and who maintains it?

Angular is a frontend framework for building SPAs. Maintained by Google and the open-source community.

Q: What is the Angular CLI command to run the app?

ng serve — runs the app on http://localhost:4200 by default.

Q: What is a SPA?

Single Page Application. The browser loads one HTML page and dynamically updates content without full page reloads.

AIM

Demonstrate flow of data between component class and template using one-way and two-way binding.

THEORY

Data binding in Angular is the automatic synchronization of data between the model (component) and the view (template).

KEY CONCEPTS

- **Interpolation (component→view):** `{{ expression }}` — embeds values into HTML text.
- **Property Binding (component→view):** `[property]="value"` — sets element properties. E.g. `[src]="imageUrl"`
- **Event Binding (view→component):** `(event)="method()"` — listens for DOM events. E.g. `(click)="fn()"`
- **Two-Way Binding:** `[(ngModel)]="property"` — 'banana in a box'. Data flows both ways.

CODE REFERENCE

```
<!-- Interpolation -->
<p>Hello, {{ userName }}</p>

<!-- Property Binding -->
<img [src]="imageUrl">

<!-- Event Binding -->
<button (click)="onButtonClick()">Click Me</button>

<!-- Two-Way Binding -->
<input [(ngModel)]="name">
<p>{{ name }}</p>
```

KEY TERMS

[Interpolation] [Property Binding] [Event Binding] [ngModel] [Two-Way]

LIKELY ORAL QUESTIONS

Q: What is two-way data binding? Which directive is used?

`[(ngModel)]` — called 'banana in a box' syntax. Data flows from component to view AND from view to component simultaneously.

Q: What is the difference between property binding and interpolation?

Interpolation embeds values in text content. Property binding sets element properties (src, disabled). Both are one-way from component to view.

Q: What syntax is used for event binding?

Parentheses: (eventName)="method()". The event flows from view to component.

AIM

Design and implement Angular forms with validation using Template-Driven and Reactive approaches.

THEORY

Angular supports two approaches for handling forms. Template-driven forms use directives in HTML. Reactive forms define structure in the component class using FormControl and FormGroup.

KEY CONCEPTS

- **Template-Driven Forms:** Logic in HTML using ngModel. Simple. Import FormsModule. Good for basic forms.
- **Reactive Forms:** Logic in TS class using FormControl, FormGroup, FormBuilder. Import ReactiveFormsModule. Scalable, testable. Best for complex forms.
- **Built-in Validators:** required, minlength, maxlength, pattern, custom validators.
- **Form States:** valid/invalid, touched/untouched, dirty/pristine.
- Two-way binding keeps form field values in sync with model automatically.

CODE REFERENCE

```
// Reactive form setup
const form = new FormGroup({
  name: new FormControl('', [Validators.required, Validators.minLength(3)]),
  email: new FormControl('', Validators.required)
});

// Template-driven (HTML)
<input [(ngModel)]="userName" required minlength="3">
```

KEY TERMS

[Template-Driven] [Reactive Forms] [FormControl] [FormGroup] [Validators] [ngModel]

LIKELY ORAL QUESTIONS

Q: What is the difference between Template-driven and Reactive forms?

Template-driven: simpler, logic in HTML, uses ngModel. Reactive: logic in TS class, uses FormControl/FormGroup, better for complex/dynamic forms.

Q: Which module is needed for reactive forms?

ReactiveFormsModule — imported in app.module.ts.

Q: What does 'dirty' mean in Angular forms?

The field value has been changed by the user (different from its initial value).

AIM

Demonstrate `*ngIf`, `*ngFor` and `*ngSwitch` for conditionally rendering HTML elements.

THEORY

Structural directives change the structure of the DOM — adding or removing elements. They are prefixed with `*` (asterisk). Angular converts the `*` syntax into an `ng-template` internally.

KEY CONCEPTS

- ***ngIf**: Adds/removes element based on boolean. Element does NOT exist in DOM when false.
- ***ngFor**: Loops over an array, repeating the element for each item. E.g. `let item of items`
- ***ngSwitch**: Like switch-case. Use `[ngSwitch]` on parent, `*ngSwitchCase` on children, `*ngSwitchDefault` as fallback.

CODE REFERENCE

```
<!-- ngIf -->
<div *ngIf="isLoggedIn">Welcome!</div>

<!-- ngFor -->
<li *ngFor="let item of items">{{ item }}</li>

<!-- ngSwitch -->
<div [ngSwitch]="color">
  <p *ngSwitchCase="'red'">Red</p>
  <p *ngSwitchCase="'blue'">Blue</p>
  <p *ngSwitchDefault>Other</p>
</div>
```

KEY TERMS

`[*ngIf]` `[*ngFor]` `[*ngSwitch]` `[Structural Directive]` `[DOM]` `[ng-template]`

LIKELY ORAL QUESTIONS

Q: What is the difference between `*ngIf` and `[hidden]`?

`*ngIf` removes the element from DOM entirely. `[hidden]` just hides it with CSS but the element still exists in the DOM.

Q: What does Angular do with the `*` in `*ngIf` internally?

Angular transforms it into an `ng-template` with a property binding: `[ngIf]='condition'`.

Q: How do you display items from an array in Angular?

AIM

Demonstrate Angular routing to navigate between components based on URL path.

THEORY

Routing in Angular refers to the mechanism of navigating between different components or views. Angular's Router module manages navigation and renders components based on the URL.

KEY CONCEPTS

- **Routes array:** Array of objects with path and component properties.
- **routerLink:** Used instead of href — prevents page reload, maintains SPA behaviour.
- **<router-outlet>:** Placeholder where routed component's view is rendered.
- **routerLinkActive:** Adds a CSS class to active navigation link automatically.
- **provideRouter(routes):** Must be added in app.config.ts to enable routing.

CODE REFERENCE

```
// app.routes.ts
const routes: Routes = [
  { path: 'first', component: FirstComponent },
  { path: 'second', component: SecondComponent },
];

// Template
<a routerLink="/first" routerLinkActive="active">First</a>
<router-outlet></router-outlet>
```

KEY TERMS

[Router] [routerLink] [router-outlet] [Routes] [SPA Navigation]

LIKELY ORAL QUESTIONS

Q: What is ?

A placeholder directive that tells Angular where to render the matched component's view when a route is activated.

Q: Why use routerLink instead of href?

href causes a full page reload. routerLink uses Angular's router for client-side navigation — no reload, SPA behaviour maintained.

Q: What are the two properties a route object must have?

AIM

Demonstrate reusing shared logic across components using Angular Services and Dependency Injection.

THEORY

A service is a class that holds shared logic, data, or HTTP calls. Dependency Injection (DI) is a design pattern where Angular automatically provides an instance of a service to a component that needs it.

KEY CONCEPTS

- **@Injectable()**: Decorator that marks a class as a service and makes it injectable.
- **providedIn: 'root'**: Registers service as a singleton available app-wide.
- **DI in constructor**: Declare service type in constructor — Angular injects it automatically.
- **CLI command**: `ng generate service serviceName`
- Services promote modularity, reusability, and maintainability.

CODE REFERENCE

```
// Creating a service
@Injectable({ providedIn: 'root' })
export class DataService {
  getData() { return [...]; }
}

// Injecting into component
constructor(private dataService: DataService) {}

fetchData() {
  this.dataService.getData();
}
```

KEY TERMS

`@Injectable` `[DI]` `[Singleton]` `[providedIn root]` `[Service]`

LIKELY ORAL QUESTIONS

Q: What is Dependency Injection?

A design pattern where Angular automatically provides (injects) dependencies to a class instead of the class creating them manually.

Q: What decorator makes a class injectable?

`@Injectable()` decorator from `@angular/core`.

Q: What does `providedIn: 'root'` mean?

The service is available app-wide as a singleton. Only one instance is created and shared across all components.

AIM

Demonstrate routing in Express.js to handle client requests based on HTTP method and URL path.

THEORY

Express is a minimal, flexible Node.js web framework. Routing determines how the app responds to a client request at a specific URI and HTTP method combination.

KEY CONCEPTS

- **Route syntax:** `app.METHOD(PATH, HANDLER)`
- **HTTP Methods:** GET, POST, PUT, DELETE, PATCH — each has a corresponding app method.
- **Route Parameters:** Named URL segments prefixed with `:` — captured in `req.params`.
- **Multiple callbacks:** Use `next()` to pass control to next handler.
- **Route paths** can be strings, string patterns, or regular expressions.

CODE REFERENCE

```
app.get('/', (req, res) => res.send('Hello World'))
app.post('/', (req, res) => res.send('POST request'))
app.put('/user', (req, res) => res.send('PUT /user'))
app.delete('/user', (req, res) => res.send('DELETE /user'))

// Route parameters
app.get('/users/:userId/books/:bookId', (req, res) => {
  res.send(req.params)
  // { userId: '34', bookId: '89' }
})
```

KEY TERMS

[Express] [Routing] [GET] [POST] [req.params] [HTTP Methods]

LIKELY ORAL QUESTIONS

Q: What is routing in Express?

Routing determines how the application responds to client requests at specific URL endpoints and HTTP methods.

Q: What are route parameters? How are they accessed?

Named URL segments prefixed with `:` (e.g. `/user/:id`). Accessed via `req.params.id`.

Q: What HTTP methods does Express support?

AIM

Demonstrate middleware functions in Express.js — their structure, purpose, and execution order.

THEORY

Middleware functions have access to req, res, and the next function. They sit between the incoming request and the final route handler, executing in the order they are registered.

KEY CONCEPTS

- Can execute any code, modify req/res, end the cycle, or call next().
- **Order matters:** Middleware loaded first executes first.
- **Must call next():** If not called and cycle not ended, request hangs permanently.
- **app.use():** Registers middleware for all routes below it.
- **Common uses:** Logging, authentication, request parsing, timestamp injection.

CODE REFERENCE

```
// Logger middleware
const myLogger = (req, res, next) => {
  console.log('LOGGED')
  next() // MUST call next() or request hangs
}

// requestTime middleware
const requestTime = (req, res, next) => {
  req.requestTime = Date.now()
  next()
}

app.use(myLogger) // register BEFORE routes
app.use(requestTime)

app.get('/', (req, res) => {
  res.send(`Time: ${req.requestTime}`)
})
```

KEY TERMS

[Middleware] [next()] [app.use()] [req] [res] [Request Pipeline]

LIKELY ORAL QUESTIONS

Q: What are the three arguments a middleware function receives?

req (request), res (response), next (function to call the next middleware).

Q: What happens if you don't call next() in middleware?

The request hangs — it never reaches the route handler. The client gets no response.

Q: How do you register middleware in Express?

Using `app.use(middlewareFn)`. It applies to all routes defined after it in the file.

Q: How is middleware different from a route handler?

Route handlers terminate the cycle with a response. Middleware processes the request and passes it forward using `next()`.

AIM

Demonstrate Create, Read, Update, Delete (CRUD) operations on MongoDB using Node.js and Mongoose.

THEORY

MongoDB is a popular NoSQL database that stores data as flexible BSON (Binary JSON) documents instead of tables and rows. Mongoose is an ODM (Object Data Modeling) library that adds schema definition, validation, and a query API on top of the raw MongoDB driver.

KEY CONCEPTS

- **NoSQL:** Not relational. No fixed schema. Documents in a collection can have different structures.
- **BSON:** Binary JSON — MongoDB's internal storage format. Supports extra types like Date and ObjectId.
- **Mongoose:** ODM for Node.js. Adds schema, validation, middleware, and clean query API.
- **Default port:** MongoDB runs on port 27017.
- **CRUD methods:** `insertOne()`, `findOne()`, `updateOne()` with `$set`, `deleteOne()`.

CODE REFERENCE

```
// Connect
const client = new MongoClient('mongodb://localhost:27017');
await client.connect();
const col = client.db('mydb').collection('users');

// CREATE
await col.insertOne({ name: 'John', age: 30 });

// READ
const user = await col.findOne({ name: 'John' });

// UPDATE
await col.updateOne({ name: 'John' }, { $set: { age: 31 } });

// DELETE
await col.deleteOne({ name: 'John' });
```

KEY TERMS

[NoSQL] [BSON] [MongoDB] [Mongoose] [ODM] [insertOne] [findOne] [updateOne] [deleteOne]

LIKELY ORAL QUESTIONS

Q: What is the difference between SQL and NoSQL?

SQL = relational, fixed schema, tables/rows. NoSQL = flexible schema, documents/key-value/graphs. MongoDB is document-based NoSQL.

Q: What is BSON?

Binary JSON. MongoDB's internal storage format. Like JSON but binary-encoded for efficiency, with extra types like Date and ObjectId.

Q: What is Mongoose and why use it over the raw MongoDB driver?

Mongoose is an ODM that adds schema definition, validation, middleware support, and a cleaner query API over the raw driver.

Q: What is the default port MongoDB runs on?

Port 27017.